

Chapitre 1 Réalisation et industrialisation

Description en code, chaînes de livraison et de provisionnement, supervision et accueil d'un locataire

Le chapitre précédent décrivait une architecture cible; celui-ci rend compte de ce qui a été construit, en distinguant partout **réalisé et vérifié, réalisé et rattaché à une vérification planifiée et écarté par une décision motivée**. Deux chaînes structurent l'exposé : la livraison applicative et le provisionnement de l'infrastructure.

1.1 Environnement de travail et description en code

Deux environnements réels correspondent à deux projets cloud distincts : le développement, dimensionné au minimum, et la recette, de topologie identique à la cible; la production est une cible documentée, non provisionnée. Séparer par projet plutôt que par étiquette rend l'isolation **structurelle** — les ressources d'un environnement ne sont pas adressables depuis l'autre — sous réserve des liaisons d'autorisation accordées à l'identité de déploiement, qu'aucun test dédié n'a éprouvées. Le développement n'est pas un miroir fidèle de la recette : la configuration du second locataire et huit variables d'accueil de plusieurs applications n'existent qu'en recette, et la chaîne de promotion n'a pas été déroulée de bout en bout. Une règle vaut cependant sans exception — *les mêmes modules servent partout, seules les variables de dimensionnement changent* — de sorte qu'aucune configuration « sans authentification en développement » ne risque d'être promue par inadvertance. Le quatrième projet d'amorçage prévu à la conception n'a pas été créé, ses éléments étant replacés dans chaque projet d'environnement : simplification cohérente avec le principe de justification (PR1), enregistrée comme écart.

Le dépôt d'infrastructure transpose le modèle en couches du chapitre 4 en un module par couche (annexe ??). Les trois politiques d'organisation prévues — pas d'adresse publique de base de données, pas de clé d'identité de service, pas d'espace de stockage public — se sont révélées inapplicables dans le périmètre administratif disponible (écart É7) : ces interdictions sont donc recherchées par la configuration des modules, les contrôles de la chaîne d'infrastructure et la revue du *plan*.

1.1.1 Frontière entre le code et l'exploitation

Le principe de description en code (PR4) s'applique à toutes les ressources durables du socle, **à six exceptions que ce mémoire nomme et justifie** : trois tiennent à la nature de l'objet, trois à une limite du dispositif et sont traitées comme des écarts, avec leur contrôle compensatoire. Les énoncer vaut mieux que revendiquer une couverture totale, car **un principe dont on ne sait pas dire la frontière n'est pas appliqué, il est proclamé**.

1. **Les valeurs des secrets et les données applicatives.** Le code crée les conteneurs, les clés de chiffrement et les droits d'accès ; jamais le contenu, qu'une description en code ferait mécaniquement apparaître en clair dans le fichier d'état.
2. **Les comptes du tableau de bord et leurs secrets de second facteur**, nés d'un appel authentifié à l'API de supervision à l'exécution ; le premier compte d'administration est créé par une tâche éphémère plutôt que par une ressource déclarée.
3. **Les enregistrements de noms de domaine**, tenus chez le bureau d'enregistrement, hors du périmètre d'administration du fournisseur cloud : dépendance externe au chapitre 4, non composant de la couche de périmètre.
4. **Le versionnement de l'espace de stockage de l'état d'infrastructure**, activé à la main. Contrôle compensatoire : la revue d'inventaire de cet espace ; condition de clôture : sa déclaration dans un module d'amorçage distinct de l'état qu'il héberge.
5. **Les ordres de révocation SQL qui séparent les locataires** sur l'instance partagée (annexe ??) : gestes d'exploitation réversibles par une restauration à un instant donné, qu'un contrôle quotidien *détecte* sans les *prévenir* (section 1.7).
6. **La liaison d'usurpation d'identité accordée à l'opérateur humain** sur l'identité de la tâche d'enrichissement, posée à la main pour rejouer les tests de bout en bout. Contrôle compensatoire : elle est nominative et figure à l'inventaire des chaînes de délégation du chapitre 4.

S'y ajoute une zone qui n'est pas une exception mais un *partage*, couture la plus délicate entre les deux chaînes et commune à toute équipe combinant infrastructure en code [**iac_morris2020**] et livraison continue : la chaîne d'infrastructure (F7) crée le service d'exécution et en possède la définition, la chaîne applicative (F3) met à jour l'image à chaque livraison, et si les deux prétendent posséder le champ « image », chaque exécution de l'une annule le travail de l'autre. **Résolution retenue** : l'infrastructure possède le service, l'applicatif possède l'image, le code d'infrastructure ignorant explicitement ce champ. Le fichier de variables de la recette n'est par ailleurs pas versionné alors qu'il porte des décisions de sécurité — chemins protégés par le filtrage applicatif, services ingérés dans l'entrepôt — ce qui limite la reproductibilité d'un provisionnement depuis un poste neuf : c'est le correctif de reproductibilité le plus rentable identifié.

1.1.2 État d'infrastructure : sécurité et détection de dérive

Le fichier d'état décrit la topologie complète du système : donnée sensible, traitée comme telle par cinq mesures détaillées en annexe ?? . C'est la règle des secrets ci-dessus qui l'empêche de devenir un coffre : aucune valeur sensible n'y a été relevée lors des revues du 19/08/2026, et son inspection exhaustive sous une identité non autorisée est l'objet du protocole T17, **conforme** le 20/08/2026 sur quatre refus opposés à trois identités de service (??). La détection de dérive [**hashicorp_drift**] repose sur une planification régulière : pour une *ressource déjà gérée dans l'état*, un écart entre la configuration déclarée et la valeur observée produit un plan non vide ; une ressource nouvelle, absente de l'état, y échappe et exige un contrôle d'inventaire complémentaire. Le protocole T16 modifie hors code un attribut réversible d'une ressource

gérée, planifie avec rafraîchissement, puis rapproche le résultat des journaux d'audit : il est porté comme partiellement conforme au chapitre 6.

Point de vigilance

Deux limites sont rapportées bien que défavorables. **Le verrouillage des versions a dû être corrigé** : le fichier de verrouillage et l'initialisation complète de la validation ont été ajoutés le 19/08/2026 après constat d'un défaut de reproductibilité. **Le plan ne prédit pas toujours l'effet réel de l'application** : la planification ne capture pas certaines validations effectuées par le fournisseur au moment de l'application, comme le montre l'incident détaillé en section 1.8.1 ; la procédure a été amendée en conséquence.

Une fois ces correctifs appliqués, la planification du 19/08/2026 sur la recette ne renvoie **aucune différence** : la configuration déclarée dans le dépôt correspond exactement à l'état réellement provisionné.

1.2 Chaîne de livraison applicative

1.2.1 Déclenchement, permissions, graphe des tâches et portes bloquantes

La chaîne applicative se déclenche sur la poussée sur la branche principale et sur l'ouverture d'une demande de fusion vers cette branche, **filtrée par chemins** (annexe ??). Le filtre n'est pas un confort : avant sa pose, **toute poussée sur la branche principale reconstruisait et redéployait la plateforme**, y compris une correction de documentation — un déclencheur non filtré dilue la signification d'un déploiement et rend illisible le lien entre un changement et son effet. Les permissions du jeton de la forge sont, elles, **déclarées au niveau du flux** — lecture du dépôt, émission d'un jeton d'identité — donc héritées par les six tâches, y compris celles d'analyse et de test qui ne s'authentifient jamais auprès du fournisseur cloud : la lisibilité d'un fichier unique l'emporte ici sur la granularité du moindre privilège. Le périmètre reste borné, aucun droit d'écriture sur le dépôt, mais l'écart est réel ; sa condition de clôture est la factorisation des deux chaînes en un flux appellable commun, sur le modèle de la chaîne dédiée à l'application pilote, qui applique déjà le réglage fin.

Les six tâches ne s'exécutent pas en séquence : elles forment un graphe orienté à **deux chemins parallèles** qui se rejoignent, celui de l'API et celui du tableau de bord, aux dépendances *bloquantes*. Vue par étapes, la chaîne de l'API en compte huit, dont trois **portes bloquantes** [**owasp_cicd_top10**] ; les tests de l'API en forment une quatrième, parallèle à l'analyse, et le chemin du tableau de bord porte la sienne (figure 1). Deux réglages absents complètent ce portrait, assumés plutôt que tus : la chaîne ne déclare ni contrôle de concurrence — deux poussées rapprochées se déploient en parallèle, sans garantie d'ordre — ni environnement protégé, donc aucune approbation manuelle avant le déploiement. Les deux relèvent du même arbitrage, une chaîne à un seul opérateur sur un environnement de recette, et de la même condition de bascule : l'ouverture de la branche principale à un second contributeur.

L'ordre des portes n'est pas arbitraire : la recherche de secrets [**gitguardian_sprawl**] précède l'analyse statique pour une raison économique, un secret étant compromis dès sa publication et sa détection à la première étape déclenchant immédiatement la rotation. Le déploiement à

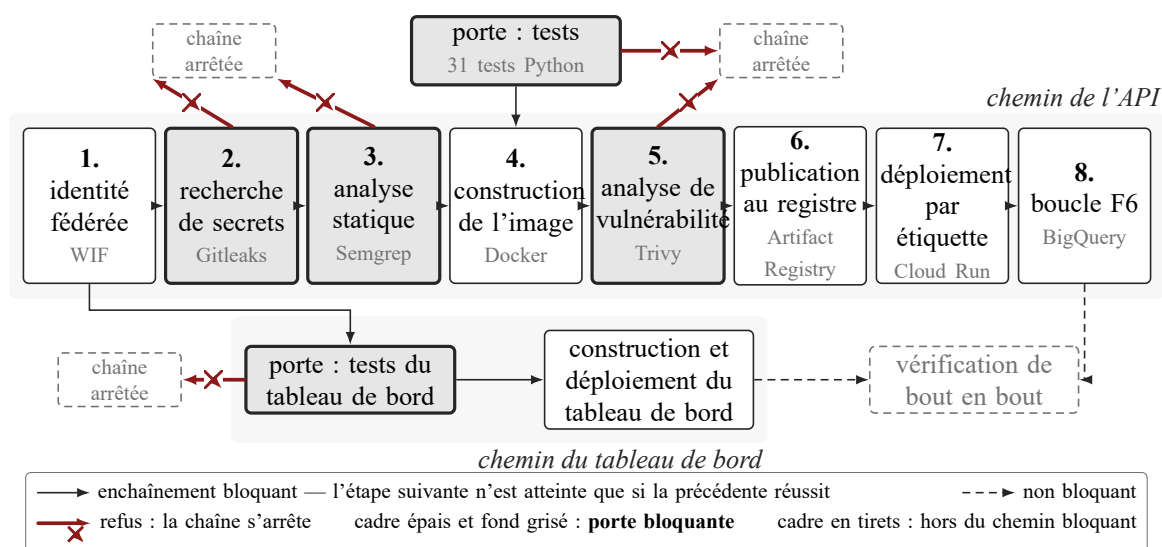


FIG. 1. Chaîne de livraison applicative. Chemin de l'API : huit étapes, dont trois portes bloquantes, plus la porte de tests qui conditionne la construction ; chemin du tableau de bord : sa propre porte. La vérification de bout en bout suit le déploiement et ne le défait pas — c'est une vérification, non une porte. Les cas de refus sont décrits en section 1.2.6 ; deux d'entre eux portent un protocole numéroté, T8 et T10, dont le chapitre 6 donne le verdict daté.

l'étape 7 s'effectue **par étiquette SHA** plutôt que par tag mutable, dans l'esprit des cadres de provenance [**slsa_framework**], et l'étape 5 s'appuie sur une analyse d'image alignée sur les pratiques recommandées pour les conteneurs [**gcp_container_best_practices**].

1.2.2 Les huit familles de contrôles

Décrire une chaîne par ses étapes ne dit pas ce qu'elle contrôle. Le tableau 1 la décrit par *familles de contrôles* ; sa dernière colonne est celle qu'il faut lire, car la destination du résultat détermine si un constat est exploitable ou seulement affiché.

TAB. 1. Les huit familles de contrôles de la chaîne de livraison applicative, dépôt vérifié le 25/08/2026.

Famille	État, outil et caractère bloquant	Destination du résultat
Recherche de secrets versionnés	Réalisée — détecteur de secrets, jeu de règles par défaut, bloquant	Journaux d'exécution de la chaîne
Analyse statique du code	Réalisée — analyseur statique, version épinglée, sept exclusions nominatives, bloquant	Journaux d'exécution de la chaîne
Analyse de la description d'infrastructure	Réalisée — analyseur de configuration, bloquant sur gravité critique	Journaux d'exécution de la chaîne
Analyse de vulnérabilité des images	Réalisée — analyseur d'image, quatre images, bloquant sur gravité critique corrigeable	Journaux et entrepôt de supervision (table des vulnérabilités) — seule famille dont le résultat quitte la chaîne
Tests automatisés	Réalisée — cadre Python (31 tests) et cadre JavaScript, bloquant sur échec, non bloquant sur couverture	Rapport de test conservé 7 jours
Analyse de composition logicielle (dépendances)	Arbitrée	Risque capté en aval par l'analyse d'image, qui inventorie les paquets de l'artefact livré ; ce chemin ne couvre ni les dépendances de développement, ni les avis sans correctif amont, écartés par configuration. Réévaluation : la première dépendance directe hors image

TAB. 1. Les huit familles de contrôles de la chaîne de livraison applicative (suite)

Famille	État, outil et caractère bloquant	Destination du résultat
Test dynamique du service déployé	Arbitrée	La vérification post-déploiement établit la disponibilité par le domaine public, jamais l'absence de vulnérabilité ; la validation dynamique du filtrage applicatif est portée par les protocoles du chapitre 6. Réévaluation : une homologation par un tiers
Signature d'artefact, provenance et nomenclature logicielle	Arbitrée (décision D07)	Traçabilité atteinte par un chemin moins coûteux — étiquette liée à l'empreinte du commit, registre unique — qui ne prouve pas que l'artefact déployé vient de la chaîne attendue. Réévaluation : l'ouverture du socle à un éditeur tiers

La boucle qui ferme le circuit. La plupart des chaînes de ce niveau s'arrêtent au constat, et le rapport meurt dans un journal que personne ne relit ; ici, le résultat de l'analyse d'image devient une donnée de supervision, interrogeable avec les mêmes outils que les détecteurs, ce qui rend possible le flux F6 (section 1.2.5). La contrepartie doit être dite : aucun constat des autres familles n'est publié au format d'échange normalisé qu'attend l'onglet de sécurité de la forge, et les vulnérabilités du tableau de bord ne sont pas exportées — la chaîne ferme *une* boucle, pas toutes.

La discipline des exclusions d'analyse statique. Chacune des sept exclusions porte une justification écrite ancrée sur un fichier et une ligne (annexe ??). Le point remarquable est ailleurs : la règle qui détecte une vérification de certificat désactivée est **volontairement non écartée**, son constat étant neutralisé par des annotations nominatives posées là où le comportement est assumé, de sorte que **toute nouvelle occurrence ailleurs dans le dépôt fasse échouer la chaîne** — une exclusion globale aurait supprimé le constat existant et, du même geste, la capacité à détecter le suivant.

Point de vigilance

Une limite de cet épinglage est reconnue et mesurée : la version de l'analyseur est figée, mais le *jeu de règles* qu'il télécharge est distant et mutable. Deux exécutions à version d'outil identique ont dénombré 612 règles le 20/08/2026 et 606 le 24/08/2026. Le verdict de la chaîne dépend donc en partie d'un référentiel que le dépôt ne contrôle pas ; la clôture de cet écart suppose de figer aussi le jeu de règles.

1.2.3 Authentification sans secret permanent

L'étape 1 réalise l'exigence EX7 par la fédération d'identité décrite au chapitre 4 : la chaîne ne détient aucune clé, et cette absence n'est pas une intention — la recherche exhaustive des ressources de clé de compte de service, des fichiers d'identifiants et de la variable d'environnement qui les désigne ne relève aucune occurrence dans le dépôt. Le point à valoriser est la forme de la condition d'attribut qui restreint cette fédération : elle porte **à la fois** sur le dépôt d'origine **et** sur la référence de branche (annexe ??). Sans la contrainte de branche, n'importe quelle branche du dépôt obtenait l'identité de déploiement et, avec elle, le droit de déployer une révision arbitraire des services. Le garde-fou qui semble couvrir ce cas, la condition de branche posée sur l'étape de construction, se trouve **côté forge** : déclaré dans le même fichier que ce qu'il

protège, un contributeur capable de modifier ce fichier le lève en même temps qu’il l’exploite. **Seul le filtre du fournisseur cloud fait autorité**, parce qu’il est déclaré ailleurs, appliqué par une autre partie et non modifiable depuis le dépôt — application directe du principe d’identité (PR2) : la décision d’autorisation appartient au vérificateur, jamais à l’émetteur.

Trois réserves complètent ce constat sans l’annuler : la liaison d’usurpation est posée sur l’attribut de dépôt et non sur celui de référence, de sorte que la restriction de branche ne tient que par la condition d’attribut ; la référence autorisée a une valeur par défaut commune aux deux environnements ; et les trois paliers d’identité prévus à l’origine ont été remplacés par une fédération et une identité de déploiement uniques par projet — le chemin d’autorisation y gagne en lisibilité, sans que l’impossibilité d’un accès croisé soit prouvée.

1.2.4 Déploiement, retour arrière et frontière avec l’infrastructure

Le déploiement de l’étape 7 remplace intégralement le trafic du service par la nouvelle révision, sans répartition progressive ; le bloc de répartition a été rendu explicite dans le code et placé hors du champ comparé par la chaîne d’infrastructure pour préparer un déploiement graduel, capacité non exploitée à ce jour. Le retour arrière est donc un geste manuel documenté — replacer la totalité du trafic sur la révision précédente, restée disponible — dont la condition de bascule est la première mise en production, où il devient un délai de rétablissement non maîtrisé. La vérification de disponibilité qui suit interroge le service par son domaine public, donc à travers le filtrage applicatif : elle éprouve la chaîne d’accès complète, non le seul conteneur.

Point de vigilance

Un défaut d’ordonnancement est rapporté ici plutôt que tu. L’image est publiée au registre *avant* l’analyse de vulnérabilité : une image porteuse d’une faille critique corrigeable entre donc dans le registre de confiance, seul son déploiement étant refusé. La chaîne écrite plus tard pour l’application hébergée construit localement, analyse, puis ne publie que si l’analyse passe — c’est l’ordre correct, et il sert de cible d’alignement.

Une réserve reste ouverte sur le schéma de données : la chaîne du socle n’automatise aucune migration, alors que celle de l’application hébergée l’exécute comme une tâche distincte, attendue jusqu’à son terme avant tout déploiement, et interdit à chaque instance de la déclencher elle-même — le bon motif, puisque N instances qui migrent sans verrou de schéma produisent une corruption silencieuse.

1.2.5 Boucle entre livraison et détection : la huitième étape

L’étape 8 réalise le flux F6, hypothèse d’apport exposée au chapitre 4 et rattachée à la question Q6 du chapitre 1 comme au verrou V4 du chapitre 2 : elle exporte le résultat de l’analyse de vulnérabilité vers une table dédiée, avec un droit d’écriture limité à cette seule table, de sorte que gravité théorique et techniques observées coexistent enfin dans un entrepôt où elles peuvent se parler. La chaîne est linéaire — analyse de vulnérabilité, service d’encodage, table des vulnérabilités, rapprochement avec les détections R1–R7 par technique ATT&CK, ordre de correction — et son apport se dit sans emphase : **F6 ne couvre aucune exigence de sécurité, il**

produit un ordre de traitement et non une réduction de risque ; sa chaîne de données existe, son utilité reste non validée (??) tant que l'ordre produit n'est pas comparé à l'ordre par gravité seule. Une limite s'y ajoute : le chargement inscrit une technique nulle pour chaque vulnérabilité, si bien que le croisement fonctionne sur la chaîne de données et non sur les valeurs. La condition de clôture est le renseignement de cette colonne à la source.

1.2.6 Refus vérifiés

Une chaîne de livraison doit être éprouvée par des cas de refus, faute de quoi rien ne distingue une porte qui laisse passer de bons artefacts d'une porte qui ne fonctionne pas — l'incident d'analyse statique de la section 1.8.1 montre que la distinction n'est pas théorique. Trois protocoles définissent ces cas : un secret factice doit arrêter la chaîne à la deuxième étape (T10), une image de base porteuse d'une faille critique connue à la cinquième (T8), et un espace de stockage volontairement exposé doit être refusé avant toute création (section 1.3). La campagne du chapitre 6 les a départagés (??) : **T8 est conforme**, sur un refus qui n'a pas été provoqué — la porte a réellement arrêté une livraison sur une vulnérabilité critique — tandis que **T10 reste partiellement conforme**, la porte étant bloquante à chaque livraison et sur l'historique complet, mais aucun refus n'ayant été opposé. Un quatrième cas, le défaut de code détecté par analyse statique, est établi autrement et mieux : par la remise en service de la porte le 19/08/2026 et sa première exécution réelle (section 1.8.1).

Un cinquième refus, non planifié, est **réellement survenu sur le code écrit pour ce projet** : la porte d'analyse de vulnérabilité a bloqué une livraison sur une faille critique embarquée par le *gestionnaire de paquets* de l'image de base, composant dont l'artefact final n'a aucun usage à l'exécution (T8, ??). Entre inscrire la vulnérabilité sur une liste d'exception et retirer la cause, c'est la seconde voie qui a été retenue, la surface d'attaque disparaissant avec l'outil qui la portait : une porte a trouvé, dans le socle lui-même, un défaut que sa conception n'avait pas anticipé.

1.3 Chaîne de provisionnement de l'infrastructure

La seconde chaîne obéit à une logique différente, et cette différence est une décision d'architecture plutôt qu'un degré d'achèvement : elle est **automatisée jusqu'au contrôle et humaine au-delà**. Son intitulé l'annonce — le fichier qui la décrit s'appelle « contrôles d'infrastructure », non « déploiement d'infrastructure ». Le déclenchement est filtré sur le répertoire d'infrastructure, selon la même règle que la chaîne applicative, puis trois étapes s'exécutent (annexe ??, figure 2).

La vérification de format est récursive et bloquante. Rendre bloquant un contrôle de forme peut surprendre ; c'est la condition d'une revue de plan utile, car une différence de mise en forme et une différence de sens se présentent à l'identique dans un écart calculé.

La validation syntaxique s'exécute sur les deux environnements, en initialisation sans accès distant : elle ne contacte ni le stockage de l'état ni le fournisseur cloud, et valide la cohérence interne de la description — références, types, arguments obligatoires, versions de fournisseurs — sans aucune autorisation sur l'infrastructure réelle. C'est ce qui permet à ce flux de ne déclarer aucune permission d'émission de jeton : il ne peut structurellement

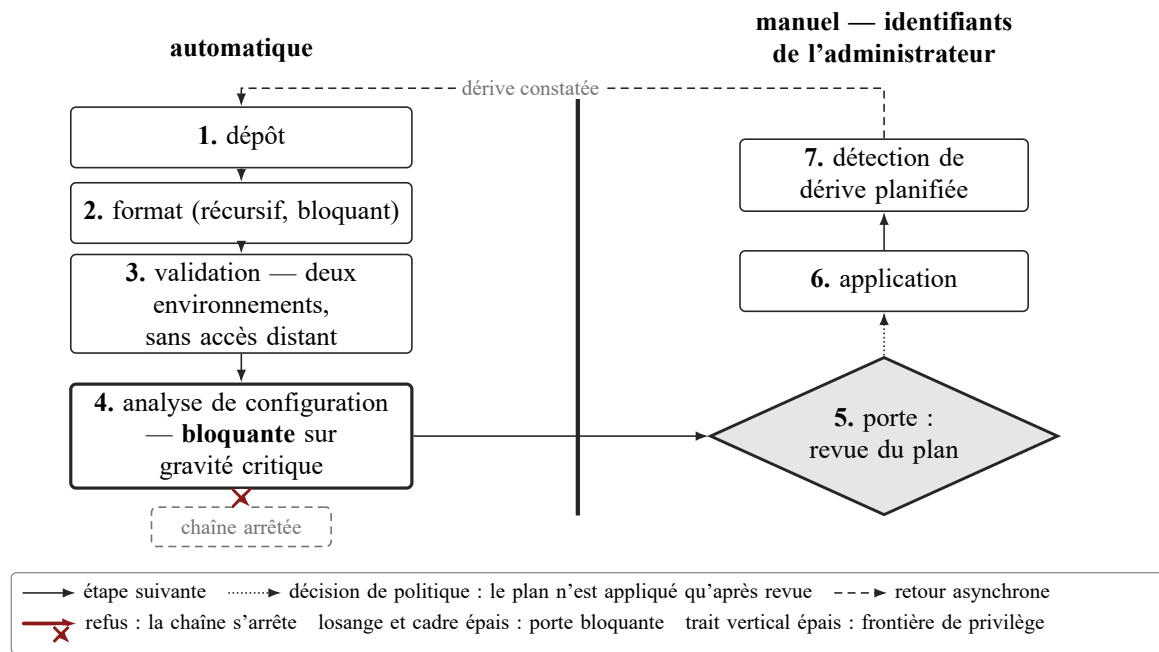


FIG. 2. Chaîne de provisionnement de l'infrastructure : la frontière automatique/manuel est un *choix de privilège*, non un degré d'achèvement — la partie automatisée n'a, par construction, aucun accès au fournisseur cloud.

pas s'authentifier auprès du fournisseur cloud, et cette impossibilité est une propriété, pas un manque.

L'analyse de sécurité de la description est une porte bloquante sur gravité critique, sur des motifs qui recoupent les trois interdictions que les politiques d'organisation auraient dû porter (section 1.1) ; un second passage, informatif, remonte les gravités inférieures sans arrêter la chaîne, de sorte que la porte n'accumule pas de dette silencieuse. **Aucun composant n'a été ajouté pour cette analyse** : c'est l'outil qui analyse les images, employé dans un autre mode — application du principe de justification (PR1).

1.3.1 La revue du plan comme porte humaine

Le plan de modification est produit puis appliqué par l'administrateur, hors de la chaîne, avec ses propres identifiants. Automatiser l'application supposerait d'accorder à l'identité de déploiement des droits de *lecture et d'écriture étendus sur tout le projet* — un plan complet lit toutes les ressources gérées, une application complète les modifie — soit la concentration de privilège que le chapitre 4 refuse.

Décision retenue. La revue humaine du plan est la porte de cette chaîne. **Condition de bascule** : l'automatisation devient rentable lorsque le périmètre justifie une *identité de déploiement d'infrastructure séparée* de l'identité de livraison applicative — c'est-à-dire à partir du troisième environnement, où la fréquence des modifications et le nombre d'opérateurs rendent le geste manuel plus risqué que le privilège qu'il évite.

Deux principes gouvernent l'ensemble. **Le plan est revu comme du code**, et se lit intégralement, jamais par sa fin : l'incident rapporté en section 1.8.1 montre qu'un plan à

périmètre restreint peut contenir un remplacement destructif sans rapport. **L'application est la seule voie d'écriture** : toute modification venue d'un autre chemin devient une dérive, détectée à la planification suivante (section 1.1.2). D'où le verdict de T18, partiellement conforme au chapitre 6 : la revue a lieu, l'approbation n'est pas outillée.

1.3.2 Protection contre les destructions

Une chaîne de provisionnement peut détruire aussi vite qu'elle construit. Deux ressources seulement portent le verrou de destruction du langage de description : les deux clés de chiffrement. Les services d'exécution, les tâches et les modules applicatifs portent explicitement le verrou ouvert, cohérent avec leur caractère reconstructible. L'instance de base de données et les tables de l'entrepôt — *y compris celles qui portent les preuves* — ne portent en revanche pas la protection de suppression du fournisseur, le seul garde-fou de l'ensemble de données étant son refus de disparaître tant qu'il contient des tables.

Point de vigilance

Cet écart est classé au premier rang du plan de correction pour un motif de proportion : le coût est de quelques minutes, et l'effacement de la table des détections par une commande d'administration les 24 et 25/08/2026 a établi que le risque n'est pas théorique. La correction pose la protection de suppression du fournisseur sur l'instance de base de données et sur les trois tables de preuve — détections, verdicts d'analyste, événements de sécurité — et sert le principe d'intégrité de la preuve (PR3), qui perd son sens si la preuve peut être détruite d'un geste.

Le quatrième scénario de refus porte sur une ressource volontairement mal configurée, dont le blocage doit être constaté avant toute création plutôt que lors d'un audit a posteriori. Sa définition est complète — ressource, politique attendue, critère d'acceptation — mais elle ne porte pas de protocole numéroté : à la différence de T8 et de T10, elle n'entre pas dans la campagne du chapitre 6, et son premier passage est porté par le plan de correction.

1.3.3 Conduite d'une migration sur un système en service

La refonte réseau arrêtée sous la décision D15 (??) a été appliquée par cette chaîne — un plan lu intégralement puis appliqué à la main à chaque étape — en douze étapes, entre le 11 et le 25/08/2026. Elle est rapportée ici parce qu'elle est la seule opération du projet à avoir déplacé le chemin du trafic d'un système déjà en service : l'ordre y compte davantage que le contenu.

Cet ordre obéit à une règle unique — *on n'interdit jamais avant d'avoir observé, et on n'observe qu'après avoir fait passer le trafic par l'endroit où on l'observe*. Les trois sous-réseaux par locataire ont donc été créés d'abord (11/08) : ajout pur, qu'aucune ressource ne référençait encore. Les charges de travail ont ensuite basculé une par une en sortie réseau directe (12 au 14/08), en commençant par la tâche planifiée d'enrichissement — dont l'échec eût été visible et sans effet sur un utilisateur — et en gardant pour la fin le service le plus exposé ; chaque bascule produit une révision, et une révision qui ne démarre pas laisse la précédente servir, de sorte que le retour arrière est un redéploiement et non une reconstruction. Le connecteur d'accès

sans serveur n'a été supprimé qu'ensuite (15/08), une fois qu'aucune ressource ne le référençait et que le plan ne présentait qu'une seule destruction.

Sept jours d'observation ont remplacé le pari par la mesure (15 au 22/08) : échantillonnage des journaux de flux porté à un paquet sur deux, journalisation de la passerelle de traduction d'adresses portée à toutes ses connexions, et construction depuis l'entrepôt de l'inventaire exhaustif des destinations réellement jointes. C'est cet inventaire, et non une hypothèse sur les besoins des charges, qui a dicté les six autorisations posées le 22/08 ; c'est la même fenêtre qui a établi que la passerelle ne traitait aucun trafic, et justifié sa suppression par une mesure.

Une seule étape était réellement dangereuse, et elle a été traitée comme telle : la pose du refus par défaut en sortie, le 23/08, qui coupe tout ce que les cinq autorisations n'ont pas nommé. Elle a été jouée hors heures ouvrées, la commande de retour arrière préparée avant l'application, et n'a été déclarée réussie qu'au terme de trente minutes d'observation. Le risque résiduel avait été nommé avant d'être couru : un flux légitime qui ne survient pas pendant la fenêtre de sept jours — une tâche mensuelle, un chemin d'erreur — n'entre dans aucun inventaire. C'est la raison d'être des refus nommés posés juste avant lui : ils font apparaître un tel flux dans les journaux sous un nom de règle lisible, au lieu de le laisser disparaître dans un refus général qui n'explique rien. La passerelle, son routeur, les deux anciens sous-réseaux vides et les trois règles d'entrée qui ciblaient une étiquette que rien ne portait ont été retirés le lendemain ; le régime permanent — échantillonnage ramené à un paquet sur dix, métadonnées exclues — a été posé le 25/08/2026.

1.4 Chaîne de détection

La collecte (flux F4) rassemble vers l'entrepôt de supervision les journaux d'audit du fournisseur, ceux des services d'exécution et les verdicts du filtrage applicatif, avec un filtrage appliqué à **la source** et non après stockage (BNF3). Une réserve porte sur les journaux d'audit : ceux d'activité d'administration sont collectés avec les autres sources, tandis que ceux d'accès aux données — entrepôt, secrets, clés, base applicative — restent dans l'espace par défaut du fournisseur, en conservation de trente jours, modifiable. L'attribution d'une action privilégiée visée par EX13 n'est donc établie que pour le plan d'administration, d'où l'exécution du protocole T13 sur les deux plans.

L'entrepôt compte dix tables (tableau 2), dont six voient leur identité d'écriture restreinte à une table unique par une liaison de droits : cette séparation vise l'exigence EX11 sans rendre les preuves immuables — une identité d'administration ou une liaison IAM excessive peut encore les modifier, ce qui doit être contrôlé séparément. Le score d'incident, recalculé à la lecture et jamais persisté (chapitre 4), est une **heuristique de tri** et non un verdict : il additionne un poids par gravité et une prime fixe lorsque les détections d'une même entité relèvent d'au moins deux tactiques distinctes, signe d'une chaîne d'attaque probable. L'asymétrie est voulue : le score est recalculé à chaque consultation et jamais conservé, le verdict de l'analyste est conservé et jamais modifié — traduction du principe d'intégrité de la preuve (PR3), **ce que la machine ordonne**

est révisable, ce que l'humain juge est définitif ; la colonne « verdict » reste vide tant que l'analyste n'a pas tranché.

TAB. 2. Modèle de données de l'entrepôt de supervision : dix tables, un écrivain chacune

Table	Contenu	Partition et rétention	Écrivain autorisé	Clé d'application
Journaux d'accès access_logs	Une ligne par requête HTTP applicative	Jour · 90 j	Normalisation F4 (identité de déploiement)	service
Journaux bruts raw_logs	Charge JSON des quatre sources collectées	Jour · 90 j	Normalisation F4	resource_name
Détections detections	Alertes R1–R7, tactique et technique ATT&CK	Jour · sans expiration	Requêtes planifiées R1–R7	service
Événements de sécurité security_events	Blocages du filtrage applicatif, déjà mitigés	Jour · sans expiration	Normalisation F4	absente
Enrichissement alert_enrichment	Technique rattachée, similarité, candidats de rang 2–3	Jour · sans expiration	Tâche F5 — droit limité à cette seule table	absente
Métriques api_metrics	Agrégat horaire : volume, erreurs, latence, blocages	Jour · sans expiration	Normalisation F4	absente
Verdicts analyst_verdicts	Jugement humain, ajout seul, jamais de mise à jour	Jour · sans expiration	API de supervision — cette seule table, rôle administrateur	absente
Vulnérabilités cve_findings	CVE du dernier scan, catalogue d'exploitation, score EPSS	Jour · sans expiration	Chaîne d'intégration — cette seule table, réécriture totale	sans objet
Vecteurs de référence attack_embeddings	872 vecteurs de 768 dimensions, catalogue ATT&CK	Aucune	Chargement hors ligne, identité d'opérateur	sans objet
File d'attente ML pending_embeddings	Déclarée, jamais écrite ni lue : vestige de F5 ; retrait décidé le 25/08/2026, non appliqué	Aucune	Aucun	sans objet

Trois colonnes portent des données à caractère personnel : l'adresse source et l'identifiant d'utilisateur des journaux d'accès, et l'entité des détections, le plus souvent une adresse. Les journaux expirent à quatre-vingt-dix jours ; les détections, qui portent la même adresse, n'expirent pas — écart assumé au titre de la valeur probante, à trancher par le responsable de traitement. Les 872 vecteurs de référence couvrent 697 techniques et sous-techniques du référentiel ATT&CK Entreprise [**mitre_attack**]. Chaque locataire dispose en outre d'un espace de stockage objet dédié, chiffré par clé gérée, protégé contre tout accès public et administrable en écriture par la seule identité de l'application concernée : il porte les **enregistrements vocaux** de l'application pilote, données biométriques qui fondent la contrainte réglementaire du chapitre 1.

Point de vigilance

La journalisation des refus du pare-feu a été activée et vérifiée le 19/08/2026, ce qui clôt l'écart É1 et rend la source de signal disponible ; elle porte, depuis la migration rapportée en section 1.3.3, sur les six règles de refus du réseau — dont les quatre refus croisés entre locataires — que le puits existant collecte sans modification. Le trajet complet d'un refus jusqu'à l'entrepôt a été mesuré depuis : le 24/08/2026, trois sorties refusées — externe, croisée entre locataires et vers la base — ont été retrouvées dans l'entrepôt en 2 min 55 s à 4 min 12 s, ce qui rend le protocole T20 **conforme** (??) ; sa cadence de rejeu est trimestrielle.

Il n'existe pas de convertisseur automatique depuis le format neutre de règles de détection vers le dialecte de l'entrepôt retenu (chapitre 2). La traduction des sept règles R1–R7 a donc été réalisée manuellement, selon un gabarit systématique — source, condition de sélection, fenêtre temporelle, agrégation, identifiants, faux positifs connus — dont l'annexe ?? donne un exemple.

La chaîne complète, de la collecte à l’affichage, est celle conçue au chapitre 4 (??) et la réalisation n’en a pas déplacé une étape : les journaux bruts alimentent les sept requêtes planifiées R1–R7 sur une fenêtre de 15 min, les détections qu’elles écrivent sont lues par l’API qui calcule le score à la requête, et la tâche d’enrichissement forme un **second cycle indépendant** — encodage puis recherche vectorielle à trois candidats — dont le résultat n’est joint à aucun score. Les deux cycles étant indépendants, une détection est affichée dès qu’elle est écrite et son enrichissement arrive au cycle suivant.

Le plafond que cette traduction manuelle impose à la couverture est établi en ?? (verrou V2) et mesuré au chapitre 6 ; la contrepartie est que les règles sont du code, versionnées, revues et déployées par la chaîne d’infrastructure.

1.5 Couche d’enrichissement sémantique

Les contrôles appliqués aux poids du modèle de représentation de phrases retenu [securebert2022, smet2023], traités comme un artefact tiers non fiable, sont ceux du chapitre 4. Ce que la réalisation ajoute est une mesure : **la fidélité est vérifiée, pas supposée**. La quantification en entiers, testée puis rejetée par la porte d’acceptation codée, dégradait les représentations au point de ne produire **aucune correspondance correcte de premier rang sur cinq cas** ; le modèle livré est donc en précision flottante simple. La tâche d’enrichissement retient trois candidats plutôt qu’un, afin que l’analyste compare des alternatives au lieu de recevoir une affirmation unique.

Posture de sortie réseau obtenue. Depuis la migration rapportée en section 1.3.3, les charges portent leurs interfaces de sortie *dans* un sous-réseau : tout leur trafic sortant traverse le pare-feu, le refus par défaut en sortie est posé, et cinq autorisations seulement le percent, chacune ciblée par une identité de service et par un port. Le plan d’adressage, les douze règles et les trois lectures qu’elles appellent sont établis en ?? ; ce que la réalisation ajoute est que **le pare-feu est devenu une source de journal nommée** — les refus croisés entre locataires et l’interdiction faite au tableau de bord de joindre la base portent chacun un nom de règle qui dit ce qui a été tenté, et rejoignent l’entrepôt par le puits déjà en place, **sans qu’aucune modification de la chaîne de détection ait été nécessaire** (annexe ??).

Cette posture n’établit pas l’impossibilité d’une exfiltration : elle en ferme la voie directe et en laisse deux ouvertes, énoncées en ?? — les interfaces du fournisseur, et les deux charges détachées du réseau. La seconde n’est pas une hypothèse : le 25/08/2026, une sonde empruntant le chemin de la révision réellement déployée du service d’encodage a atteint une destination externe en 412 ms, quand la même sonde passant par le sous-réseau était refusée deux fois. **Le protocole T14 est donc non conforme** ; le correctif — réintégrer les deux charges au sous-réseau sous une étiquette qu’aucune autorisation de sortie ne couvre — coûte une révision sans changement applicatif. Le trajet d’un refus jusqu’à l’entrepôt est, lui, mesuré et **conforme** par T20 (24/08/2026, ??).

1.5.1 État d’avancement réel de la couche

L’écart entre conception et réalisation est ici le plus important (tableau 3).

TAB. 3. État d'avancement de la couche d'enrichissement sémantique

Composant	Réalisé	Statut
Préparation et figement du modèle	Oui	Vérifié
Chemin public direct absent dans la configuration	Oui — mais la sortie <i>managée</i> de la révision déployée reste hors du pare-feu	T14 non conforme (25/08/2026)
Table de vecteurs de référence et index	Oui	Vérifié
Tâche d'enrichissement, recherche vectorielle	Oui	Réalisé
Cadence prévue (5 min)	Non — 15 min	Écart assumé (coût)
Cache par étiquette SHA d'entrée	Calculé, jamais exploité	Non réalisé
Vue de corrélation entre alertes	Non	Non réalisé
Enrichissement dans le score d'incident	Non	Écart majeur

Point de vigilance

La couche d'enrichissement produit ses résultats, mais ceux-ci n'alimentent pas le score d'incident présenté à l'analyste. Cet écart ne remet pas en cause la démonstration technique — la chaîne complète fonctionne et le chapitre 6 la mesure — mais il borne l'affirmation possible : le rattachement automatique produit des candidats consultables sans preuve suffisante de précision, de coût stabilisé ni d'amélioration de la priorisation. **Décision du 25/08/2026** : le positionnement informationnel est assumé comme état livré, et la jointure au score est reportée en perspective. Elle ne se réduit pas à une jointure technique : elle suppose une règle de décision évaluée et surtout l'ajout d'une clé d'entité à la table d'enrichissement, dont le chapitre 4 montre qu'elle est absente du schéma.

1.6 Tableau de bord et API de supervision

La supervision repose sur deux logiciels écrits pour ce projet : le tableau de bord, application web à rendu serveur, et l'API de supervision qui l'alimente (figure 3). Tout passe par l'API, qui applique le contrôle d'autorisation ; cette séparation réalise les exigences EX5 et EX6 et évite l'erreur fréquente donnant à une interface web les droits de lecture d'un entrepôt. Les vues se rafraîchissent en continu et signalent la fraîcheur de leurs données, mais cette actualisation porte sur des résultats *déjà calculés* : l'interface est vive, la détection ne l'est pas au même degré.

1.6.1 La chaîne de données d'une vue

Le point le plus solide de cette architecture n'est pas visible à l'écran : **aucune vue du tableau de bord n'interroge directement l'entrepôt de supervision ni la base applicative.** Toutes empruntent les cinq mêmes sauts, sans exception vérifiée ligne à ligne (figure 4, extrait en annexe ??) : cette uniformité est ce qui rend la chaîne auditable.

Aucun script exécuté dans le navigateur ne peut donc lire le jeton, ni l'exfiltrer, y compris en cas d'injection de script réussie dans une vue : **le tableau de bord n'est pas un client privilégié, c'est un client comme un autre, qui n'a de pouvoir que celui du jeton qu'on lui a confié.** Les sept routes serveur ne dérogent pas à ce motif : ce sont des relais d'authentification, qui lisent le témoin, en font un en-tête d'autorisation, transmettent et rendent la réponse. L'une d'elles porte une décision : le relais de connexion *préserve* le code de refus pour dépassement de seuil au lieu de le réécrire en refus d'authentification, ce qui aurait fait croire à l'analyste qu'il s'était trompé de mot de passe et privé le centre d'opérations du signal d'attaque. Le garde de session vérifie la présence du témoin et sa non-expiration par simple décodage de la date d'échéance, **sans**

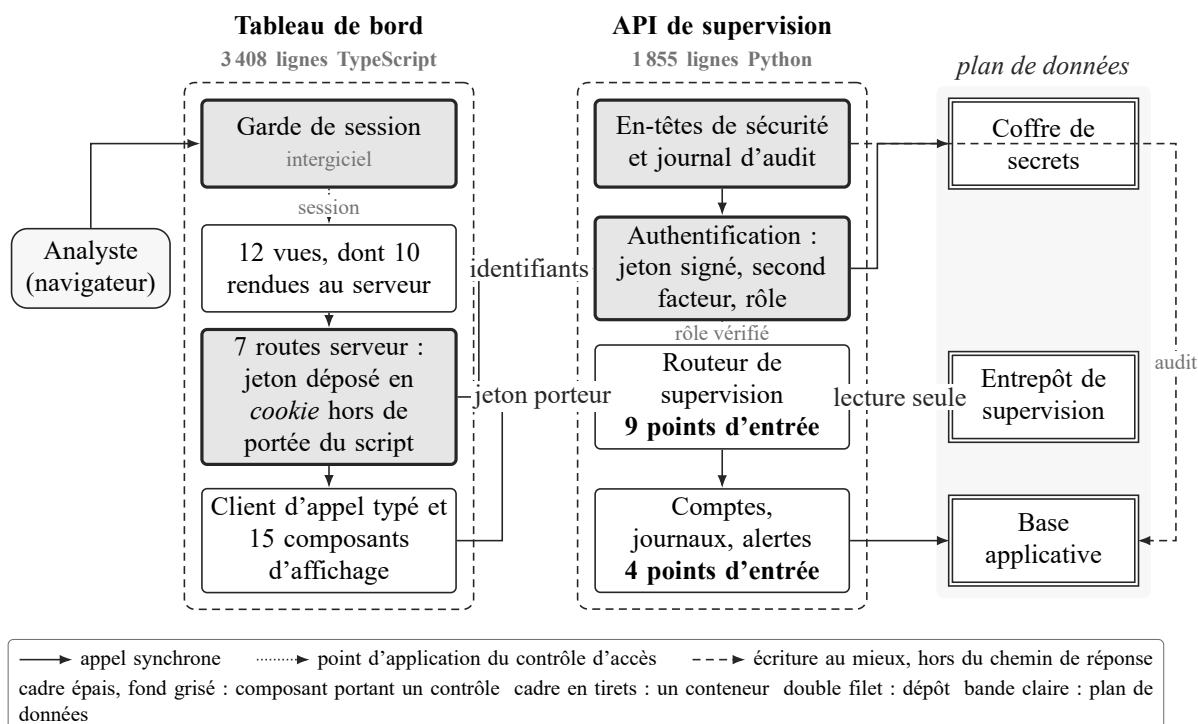


FIG. 3. Architecture interne des deux logiciels écrits : composants, frontières de déploiement et points d'application du contrôle d'accès. Le rôle est vérifié sur les treize points d'entrée de données de l'API — neuf de supervision et quatre de gestion — et jamais dans le tableau de bord, qui ne détient aucune autorisation propre ; l'échec de l'écriture d'audit est absorbé et ne fait jamais échouer la requête de l'analyste.

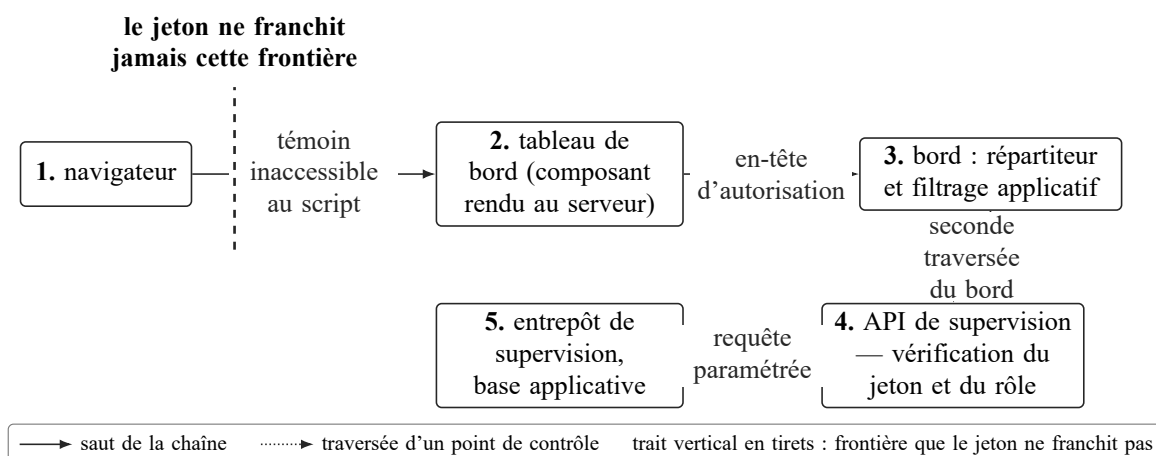


FIG. 4. Chaîne de données d'une vue du tableau de bord : les cinq sauts, et la frontière que le jeton ne franchit pas. L'autorité d'autorisation est l'API de supervision, qui vérifie le jeton et le rôle ; l'identité du tableau de bord ne détient aucune autorisation sur les données.

vérification de signature — le tableau de bord ne connaît pas la clé et ne doit pas la connaître ; ce contrôle est *documenté dans le code comme un contrôle d’ergonomie et non d’autorisation*, la seule source de vérité restant l’API.

1.6.2 Cartographie des vues

Le tableau 4 met en regard, pour chaque vue, ce qu’elle apporte à l’analyste avec le point d’entrée qu’elle appelle, et sa capacité à être filtrée par locataire.

TAB. 4. Cartographie des vues du tableau de bord : douze éléments rendus, dont dix au serveur, dépôt vérifié le 25/08/2026.

Vue	Apport à l’analyste et point d’entrée appelé	Filtre locataire
Vue d’ensemble	Cinq indicateurs, répartition par gravité, courbe des requêtes, dix dernières requêtes — journaux, alertes et synthèse en parallèle	Partiel — synthèse seule
Journal d’audit	Cent dernières requêtes reçues par l’API — journaux	Non
Alertes	Requêtes en erreur, gravité dérivée du code de retour — alertes, filtrées	Non
Détections	Deux cents détections des règles R1–R7 sur vingt-quatre heures — détections	Oui
Incidents	Entités agrégées, score de tri, verdict posé — incidents	Oui
Fiche d’incident	Jauge de score, tactiques rattachées, chronologie — incident par entité, 200 lignes	Oui
Couverture ATT&CK	Matrice de couverture par tactique — couverture	Oui
Vulnérabilités	Priorisées : exploitation connue, observation, score de probabilité — vulnérabilités	Non
Santé des règles	Rendement et taux de faux positifs des sept règles — santé des règles	Non
Enveloppe de mise en page	Navigation, bandeau, bascule de thème, indicateur de fraîcheur — aucun appel	Sans objet
Connexion (<i>cliente</i>)	Formulaire en deux temps, mot de passe puis code temporel — relais de connexion puis de vérification	Sans objet
Sécurité du compte (<i>cliente</i>)	Enrôlement et retrait du second facteur — relais du second facteur	Sans objet

La colonne de droite est presque toujours mal lue : la portée du filtre par locataire n’est pas un défaut d’interface qu’un développement complémentaire viendrait combler, c’est une **propriété du modèle de données**. Une vue ne peut être filtrée par locataire que si la table qu’elle interroge porte une clé de locataire fiable ; or trois tables seulement de l’entrepôt en portent une, et une seule de ces trois clés est renseignée sur la totalité des lignes (tableau 2). Les vues d’ensemble, de vulnérabilités et de santé des règles restent donc globalement fusionnées quel que soit le locataire sélectionné, ce que le sélecteur affiché à l’écran documente lui-même. La condition de bascule est l’introduction d’une clé de locataire obligatoire à l’ingestion, préalable de tout cloisonnement au stockage.

Point de vigilance

Le filtre par locataire est porté par un témoin déposé côté navigateur, non signé, et le jeton ne porte aucune revendication de locataire. Tout analyste authentifié peut donc changer de locataire ou retirer le filtre. Le mémoire l’énonce comme le code l’énonce : **c’est un raffinement de lecture, pas une frontière de sécurité**. Le transformer en frontière suppose une revendication de locataire dans le jeton et un filtre imposé côté serveur — travail identifié, chiffré, et subordonné au préalable de la clé de locataire.

1.6.3 Authentification, second facteur et contrôle d'accès

Trois rôles sont définis dans le jeton émis par l'API : administrateur, analyste et service — cette dernière identité, non humaine, n'étant référencée par aucun contrôle à ce jour ; aucun rôle n'est prévu pour les utilisateurs de l'application hébergée, qui n'accèdent jamais au tableau de bord. L'écriture d'un verdict illustre l'exception prévue au principe d'intégrité de la preuve (PR3) : elle est possible, mais uniquement dans une table qui lui est propre (tableau 2), jamais dans les tables de preuve, et réservée à l'administrateur — arbitrage plus restrictif que la répartition des tâches d'un centre d'opérations, cohérent avec la règle « une seule identité écrit dans une table donnée », et dont la condition de réévaluation est l'arrivée d'un second analyste. L'interface a été servie à un compte analyste le 23/08/2026 sur des détections réelles de la recette, une alerte R2 y apparaissant rattachée à la technique T1498 et attribuée au locataire concerné.

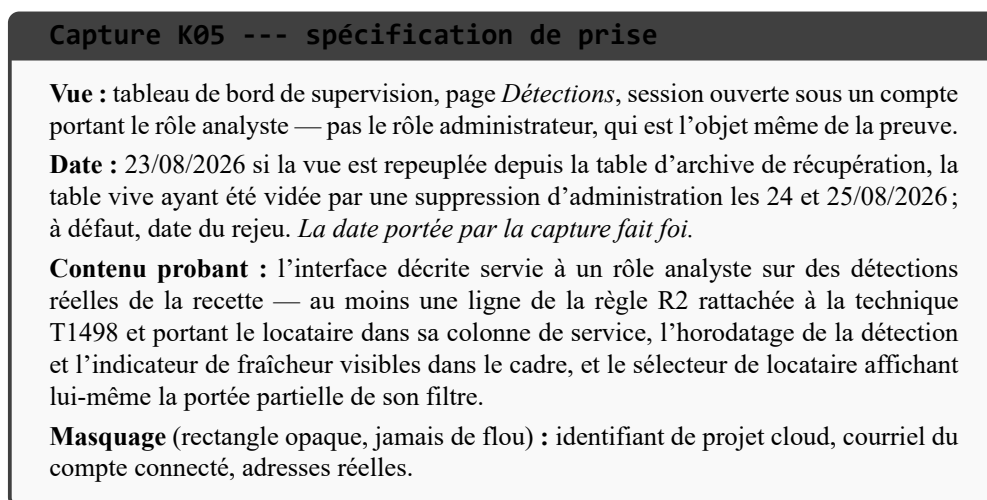


FIG. 5. Tableau de bord de supervision servi à un compte de rôle analyste, page des détections. À y lire : une détection de la règle R2 rattachée à la technique T1498 et attribuée au locataire, l'horodatage de la détection porté par la capture elle-même, et le sélecteur de locataire qui énonce la portée partielle de son filtre.

Le facteur de connaissance. Une longueur minimale de **douze caractères** est imposée par le schéma de validation d'entrée et non par un contrôle écrit à la main : la contrainte est portée par le contrat de l'API et vaut pour tout appelant, sans exigence de complexité, d'expiration ni d'historique, conformément aux recommandations actuelles. La vérification borne en outre la longueur de l'entrée avant la fonction de hachage, faute de quoi un appel *non authentifié* portant un mot de passe très long provoquait une erreur interne au lieu d'un refus — défaut de disponibilité atteignable sans compte, corrigé.

La session. Deux propriétés de la vérification du jeton signé décrit au chapitre 4 méritent d'être relevées. L'algorithme de signature est **figé en constante** et n'est jamais lu depuis l'en-tête du jeton présenté, ce qui immunise contre la confusion d'algorithme — la classe d'attaque où un jeton signé par un autre algorithme, voire par aucun, est accepté parce que le vérificateur croit ce que le jeton déclare de lui-même. La validation exige par ailleurs la présence effective des champs d'échéance et d'émission, un jeton dépourvu d'échéance ayant été auparavant valable indéfiniment.

Le second facteur. La séparation du jeton intermédiaire et du jeton d'accès, conçue au chapitre 4, est **vérifiée à chaque appel** : la dépendance qui identifie l'appelant refuse tout jeton dont le type n'est pas celui d'un jeton d'accès. Le point d'entrée qui reçoit le code temporel n'était par ailleurs couvert que par un limiteur applicatif, non par la limitation de débit du périmètre ; il a été ajouté aux chemins d'authentification protégés le 19/08/2026 et vérifié en conditions réelles le jour même : onze requêtes successives ont produit dix refus de validation, puis un refus pour dépassement de seuil. C'est cette trace qui étaye le test T2.

L'anti-force-brute repose sur deux couches, et le choix de la clé de comptage est un raisonnement d'ingénieur plutôt qu'une configuration. La couche principale est au bord : le filtrage applicatif limite les appels aux chemins d'authentification par adresse source réelle, avec bannissement temporaire au dépassement. La couche applicative ne peut pas utiliser cette clé, le tableau de bord et l'API partageant une **adresse de sortie commune** : lorsque la requête est relayée par le tableau de bord, la seule adresse que voit le limiteur applicatif est celle de cette sortie, de sorte qu'une clé par adresse rendrait le compteur *commun à tous les analystes*. La conséquence est quantifiable : douze requêtes non authentifiées suffisaient à verrouiller le centre d'opérations entier, en refusant la connexion à des analystes qui n'avaient rien fait. La clé porte donc sur le **défi** — le jeton intermédiaire remis à l'issue du mot de passe — ce qui isole chaque tentative dans son propre budget d'essais. Le code énonce lui-même la limite résiduelle : rejouer l'authentification par mot de passe réobtient un jeton intermédiaire, donc un budget, ce qui reporte la charge sur la couche de bord, laquelle voit l'adresse réelle du client. La leçon dépasse ce cas : **un contrôle qui suppose qu'une adresse réseau désigne un utilisateur produit des blocages collectifs dès qu'une agrégation existe entre lui et le client** — la question n'est pas « quel seuil ? » mais « que compte-t-on, et cet objet est-il propre à celui que l'on veut ralentir ? ».

Le secret partagé du second facteur était auparavant stocké en clair. Un **chiffrement authentifié applicatif** a été mis en place le 19/08/2026, avec migration non destructive tolérant les valeurs antérieures et clé conservée au coffre de secrets ; la migration des valeurs déjà en base n'étant pas documentée comme exécutée, l'écart n'est pas clos.

1.6.4 Le logiciel écrit : structure, tests et dette assumée

Les deux composants de cette interface sont les seuls éléments du socle écrits ligne à ligne plutôt que déclarés (tableau 5). Le contrôle d'autorisation conditionne la portée de l'exigence EX5 et appelle une description exacte : les neuf points d'entrée de supervision déclarent tous le *même* composant, qui valide le jeton, en extrait le rôle et refuse l'appel si ce rôle n'est pas attendu — huit admettent l'administrateur et l'analyste, le neuvième, celui qui enregistre un verdict, n'admet que l'administrateur. Il n'y a donc pas neuf contrôles à auditer mais un seul, déclaré neuf fois, ce qui *déplace* le risque : un point d'entrée ajouté sans la déclaration serait ouvert sans qu'aucun test existant ne s'en aperçoive. Le remède, un test d'inventaire vérifiant que toute route déclarée porte son contrôle, est porté au plan de durcissement de l'API.

Deux lignes de ce tableau sont défavorables et appellent leur cause. La couverture de 3,00 % tient à ce que les vues ne sont pas testées : seules des fonctions pures le sont, et deux des quatre suites éprouvent du code recopié dans le fichier de test plutôt que le module lui-même —

TAB. 5. Caractérisation quantifiée des deux logiciels écrits, dépôt vérifié le 25/08/2026.

	API de supervision	Tableau de bord
Langage et cadre	Python 3.12, cadre applicatif asynchrone à validation typée	TypeScript, cadre à rendu serveur
Volume écrit	1 855 lignes, 20 modules	3 408 lignes, 40 fichiers
Points d'entrée	21, dont 9 pour la supervision	douze vues, dont dix rendues au serveur et deux clientes, et sept routes serveur
Modèle de données	4 tables, 3 migrations, 22 schémas de validation	16 types partagés avec l'API
Tests automatisés	31 tests, exécutés à chaque livraison	22 tests unitaires exécutés ; 22 tests de parcours écrits mais jamais exécutés par la chaîne
Couverture mesurée	non mesurée par la chaîne : l'outil est installé, l'option de mesure ne l'est pas	3,00 % des lignes — 11 lignes couvertes sur 367, 22 fichiers instrumentés — pour un seuil de 70 % déclaré et non appliqué
Dette assumée	aucune version portée par les points d'entrée : un client ne peut pas se lier à un contrat stable	375 lignes d'une interface antérieure, non déployée, conservées au dépôt

défaut de méthode plus que défaut de couverture. Le progrès réel est ailleurs : la chaîne *exécute* désormais ces tests, en porte bloquante avant la construction. L'absence de version sur les points d'entrée est sans conséquence tant que le tableau de bord en est le seul client, et sa condition de réévaluation est l'ouverture à un client tiers.

Deux comportements de repli sont visibles à l'exécution et doivent être nommés. L'intergiciel qui écrit le journal d'audit **absorbe toute exception** : une panne d'écriture ne fait jamais échouer la requête de l'analyste. L'arbitrage est délibéré — aveugler le centre d'opérations pendant que la base est en difficulté serait pire — mais il entre en tension avec l'intégrité de la chaîne de preuve, une défaillance d'écriture ne se signalant par rien ; le contrôle compensatoire est identifié, **une alerte sur l'absence d'écriture**, déclenchée lorsque le compteur de lignes d'audit tombe à zéro alors que le service reçoit du trafic. Le tableau de bord **bascule sur un jeu de données de démonstration** lorsque l'API est injoignable : repli signalé par un bandeau visible, désactivé sur le flux d'authentification — aucune session ne peut donc être ouverte sans l'API — et autorisé hors d'une construction de production seulement. Comme il porte un catalogue de règles *fictif*, la règle s'énonce sans détour : aucune démonstration hors de l'image de production, la condition de clôture étant l'alignement du catalogue de repli sur les sept règles réelles.

1.7 Accueil d'un locataire et frontière de responsabilité

Le besoin BF13 exigeait que le socle accueille une nouvelle application sans reconception. Le module d'exécution des services applicatifs a donc été rendu paramétrique en cours de projet : le nom de l'application est une variable, et chaque application reçoit sa propre identité, ses propres secrets et son propre service. Le socle héberge la plateforme MENAL elle-même, API et tableau de bord, et à côté d'elle l'application pilote ELSON du chapitre 1 : c'est cette seconde instance, « le second locataire », qui sert de preuve d'accueil.

1.7.1 La procédure d'accueil et la portée de l'isolation

La procédure a été reconstituée à partir du cas réellement déployé, et non écrite *a priori* : chaque étape correspond à un geste qui a dû être fait. Elle suppose de l'application quatre prérequis non négociables — aucun état sur le disque local, configuration par variables d'environnement, un secret par usage, écoute sur le port fourni avec un point de santé sans

dépendance externe — et se déroule ensuite en six étapes (figure 6). Quatre d’entre elles portent une contrainte que le cas réel a imposée. L’enregistrement de nom de domaine doit exister *avant* tout provisionnement, faute de quoi le certificat géré reste indéfiniment en cours d’émission, et les images doivent avoir été publiées une première fois (étape 1). L’application se fait dans l’ordre, à périmètre restreint d’abord — identité, secrets, base, espace média — puis complète, le plan étant lu intégralement à chaque fois (section 1.8.1, étape 4). La chaîne de livraison dédiée de l’étape 5 est recopiée depuis celle de l’application pilote : cette duplication est l’écart le plus coûteux de la procédure, et sa correction, un flux appelable commun, est identifiée. La quatrième contrainte est la plus traître : les cinq variables d’environnement de l’étape 3 — services collectés, chemins d’authentification protégés, routage au bord, services supervisés, exclusions de règle — doivent être mises à jour *ensemble*, chacune remplaçant un repli historique ; renseigner l’une sans y réinclure les valeurs existantes coupe la collecte ou l’anti-force-brute de la plateforme en croyant ajouter une application.

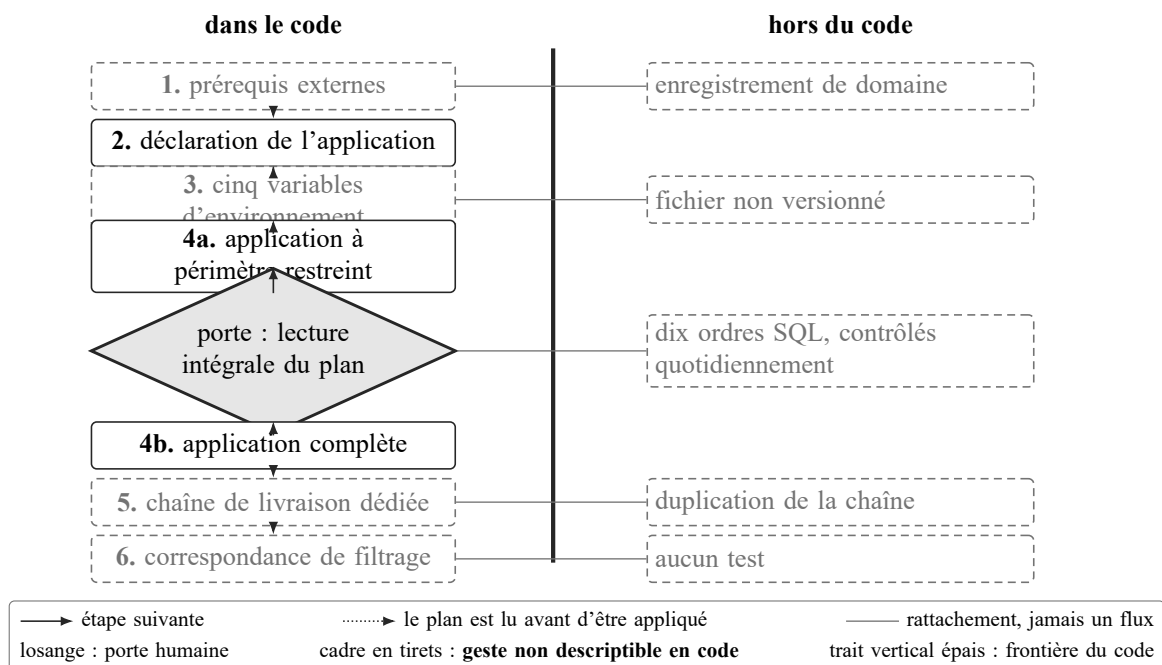


FIG. 6. Accueil d’un locataire : six étapes, et les cinq gestes qui restent hors du code. L’accueil est reproductible sur le plan de l’identité et du déploiement ; les cinq gestes non descriptibles en code sont la mesure exacte de la maturité multi-locataire du socle.

Point de vigilance

S’ajoute à ces six étapes un geste non descriptible en code en l’état : **dix ordres SQL, dont six révocations** qui retirent aux comptes applicatifs l’appartenance au rôle privilégié de l’instance (annexe ??). Un contrôle quotidien vérifie qu’ils tiennent toujours : **il détecte la dérive, il ne la prévient pas**. Le chemin par lequel cette frontière se défait n’est pas la restauration elle-même mais la réconciliation qui la suit, et il n’a aucune borne temporelle : le raisonnement complet, sa conséquence sur la conception et la condition de clôture sont établis en ??.

Le second locataire a été accueilli en recette le 07/08/2026 par instanciation du module d'exécution paramétrique : compte de service dédié à deux rôles (client base de données, écriture de journaux), base de données et utilisateur séparés, DNS et certificat gérés, six alertes et une sonde de disponibilité; depuis le 08/08/2026, les secrets et l'espace média sont chiffrés par clé gérée dédiée. Le contrôle quotidien d'isolation du 19/08/2026 a produit **six vérifications réussies sur six**, la connexion croisée entre bases étant effectivement refusée.

L'isolation obtenue porte sur quatre plans, dont un seul reste partagé, et cette délimitation vaut mieux qu'une appréciation globale. **L'identité et les secrets** sont isolés sans réserve : le compte de service de l'application hébergée ne détient *aucun droit sur l'entrepôt de supervision*, ce qui lui interdit structurellement de lire les données de sécurité, les siennes comprises. **Les données** sont séparées logiquement — base et utilisateur dédiés — sur une instance partagée, par les ordres décrits ci-dessus. **Le réseau est isolé depuis la migration du 11 au 25/08/2026** (section 1.3.3) : un sous-réseau par locataire, et quatre refus nommés — deux en sortie, deux en entrée — interdisent chaque sens de traversée et journalisent toute tentative sous un nom de règle exploitable comme signal de mouvement latéral. Le connecteur unique, point de défaillance commun aux deux locataires, n'existe plus.

Le plan qui reste partagé est celui du stockage. Les deux locataires demeurent sur *une seule* instance de base de données et *un seul* jeu de tables de supervision : la ségrégation réseau empêche une identité d'atteindre les charges de l'autre locataire, non les deux d'atteindre le même serveur, puisque les deux règles qui l'autorisent visent la même adresse ; le contrôle reste au niveau du moteur de base. L'isolation des données est conçue, chiffrée et ordonnancée en ??, et non appliquée — la raison en est écrite plutôt que tue : **deux migrations d'infrastructure dans la même quinzaine, conduites par un opérateur unique, ne seraient pas soutenables.**

Point de vigilance

La portée exacte de cette frontière est celle établie en ?? : le sélecteur disponible étant l'étiquette réseau et non le compte de service, elle est **opposable à une charge de travail compromise, non à un opérateur capable de déployer**, qui poserait l'étiquette de l'autre locataire. Ce second cas relève du resserrement du droit de déploiement (??), non du réseau ; et la conception ne tient pas au-delà de trois ou quatre locataires, les refus croisés croissant comme le carré de leur nombre.

Le seuil de bascule reste l'**accueil d'un troisième locataire** (??), mais son contenu a changé : sur les quatre travaux qu'il commandait, **deux sont faits** — la sortie par locataire, obtenue par un sous-réseau propre plutôt que par un connecteur dédié, et les règles internes ciblées par identité, portées par la convention d'étiquetage ci-dessus. Les **deux autres subsistent** : une clé de locataire obligatoire à l'ingestion et un cloisonnement au stockage de l'entrepôt, auxquels s'ajoute l'instance de base dédiée au locataire porteur de donnée irréparable (??). Tous relèvent du plan de la donnée, aucun du réseau ; le sous-réseau du troisième locataire est déjà créé et adressé, de sorte que son accueil est devenu, côté réseau, une ligne de configuration.

L'expérimentation montre la réutilisabilité partielle du module d'exécution sans qualifier le socle de produit multi-locataire abouti, et son coût de sécurité se dit symétriquement : le périmètre d'usurpation accordé à l'identité de déploiement croît avec chaque application

accueillie. Un dernier piège appartient à la pire catégorie, celle des défaillances silencieuses : omettre le nouveau service dans la liste des sources collectées rend l'application *absente de l'entrepôt sans produire la moindre erreur* — l'application tourne, le socle la sert, et le centre d'opérations ne la voit pas.

1.7.2 La frontière de responsabilité

Héberger l'application d'un tiers, fût-il du même groupe, oblige à trancher qui répond de quoi. Le tableau 6 énonce cette frontière sur quatorze domaines ; les cases où le socle n'apporte rien y figurent au même titre que celles où il apporte tout, et un domaine reste laissé à découvert par les deux parties — c'est le plus utile à lire.

TAB. 6. Frontière de responsabilité entre le socle et l'éditeur de l'application hébergée, état au 25/08/2026.

Domaine	Le socle (hébergeur)	L'éditeur de l'application hébergée
Noms de domaine, transit, certificats	Intégral : routage par hôte, certificat géré, politique de chiffrement, transport strict	—
Filtrage applicatif au bord	Intégral : politique unique, limitation de débit, blocage géographique, motifs connus	Déclare ses chemins d'authentification à protéger
Réseau — périmètre et sortie	Intégral : entrée par le seul répartiteur ; en sortie, refus par défaut, cinq autorisations nommées par identité et par port, refus journalisés vers l'entrepôt. <i>Réserve</i> : les interfaces du fournisseur restent une destination autorisée et non inspectée	—
Réseau — segmentation entre locataires	Intégral <i>au plan réseau</i> depuis le 25/08/2026 : un sous-réseau par locataire, refus croisés nommés et journalisés dans les deux sens, plus de connecteur partagé (T20). <i>Réserve</i> : opposable à une charge compromise, non à un opérateur capable de déployer ; deux charges à sortie managée hors du pare-feu (T14) ; instance de base et entrepôt partagés	Déclare ses charges, qui reçoivent l'étiquette de leur compte de service et le sous-réseau de leur locataire
Exécution des conteneurs	Intégral : plateforme, quotas, sondes, mise à l'échelle, révisions, entrée restreinte au répartiteur	Image conforme : exécution non privilégiée, port imposé, point de santé sans dépendance externe
Identité machine et secrets	Intégral : compte de service dédié à moindre privilège, usurpation ciblée, un secret par usage, clé gérée	Consomme les secrets par référence, ne les journalise jamais
Base de données — instance, sauvegarde, restauration	Intégral : instance régionale, adresse privée, restauration à un instant donné ; clé gérée structurellement inapplicable, le champ étant immuable après création. <i>Réserve</i> : une restauration par clonage porte les deux bases	Cohérence applicative après restauration, données hors base
Base de données — schéma et isolation logique	Partiel : vérifie quotidiennement que la séparation tient	Applique le durcissement ; geste non descriptible en code à ce jour
Stockage des médias et chiffrement au repos	Intégral au niveau de l'infrastructure : espace dédié, accès public interdit, clé gérée	Sert les objets, gère les accès temporaires, et répond du chiffrement applicatif des données sensibles , aujourd'hui absent
Code applicatif, logique métier, validation des entrées	Aucune sur la logique ; le filtrage au bord n'arrête que des charges <i>connues</i>	Intégral
Identités des utilisateurs finaux	Aucune : le second facteur du socle ne couvre que les comptes du tableau de bord	Intégral : authentification, autorisation et second facteur des utilisateurs de l'application
Journalisation, détection et notification	Journalisation d'infrastructure, normalisation, conservation, R1–R7, rattachement ATT&CK, enrichissement. <i>Réserve</i> : la journalisation métier est ingérée sans être exploitée, et l'éditeur n'est pas notifié	Émet ses journaux, déclare ses exclusions de règle, conserve son alerte fonctionnelle
Chaîne de livraison et analyse d'image	Intégral : authentification sans clé, registre unique, analyse de vulnérabilité, restitution à l'entrepôt	Corrige les vulnérabilités remontées

TAB. 6. Frontière de responsabilité entre le socle et l'éditeur (suite)

Domaine	Le socle (hébergeur)	L'éditeur de l'application hébergée
Conformité, base légale, droits des personnes	Aucune : des moyens techniques, jamais une gouvernance	Intégral : base légale, information des personnes, exercice des droits, durées de conservation

Le socle sécurise le bord, le réseau, l'exécution, l'identité machine et la chaîne de livraison. La logique métier, les identités des utilisateurs finaux et la conformité restent à la charge de l'éditeur. Cette frontière n'est pas un partage de commodité : elle correspond à ce qu'un hébergeur peut techniquement contrôler.

Une ligne appelle une lecture particulière, car elle est vide des deux côtés : le chiffrement applicatif des données sensibles. Le socle chiffre les supports, il ne chiffre pas les champs, et l'éditeur ne l'a pas fait. Ce vide n'est pas un oubli de rédaction ; c'est l'exigence qu'un contrat d'hébergement devrait porter en premier, et le mémoire la nomme plutôt que de la répartir arbitrairement. La segmentation réseau entre locataires occupait cette même case jusqu'à la migration du 11 au 25/08/2026 (section 1.3.3) : elle a basculé côté hébergeur, seul domaine du tableau à avoir changé de camp. **Un domaine sort du découvert lorsqu'une partie l'assume dans son code, non lorsqu'un contrat le lui attribue.**

1.8 Conduite du projet

J'ai conduit seul toutes les étapes du projet : audit du dépôt existant, rédaction des exigences, conception de l'architecture, écriture des modules d'infrastructure, construction des deux chaînes, traduction des règles de détection, mise en place de la couche d'enrichissement et du tableau de bord, puis exploitation quotidienne du socle en recette — celle qui a produit les cinq incidents ci-dessous. Un assistant de programmation a servi à produire des squelettes répétitifs, des tests et des relectures, jamais à décider d'une option d'architecture ou de sécurité, et aucune production assistée n'a été retenue sans exécution : elle est passée par les portes de la chaîne de livraison puis par une vérification en conditions réelles. Les deux changements de périmètre significatifs — abandon du quatrième projet cloud d'amorçage (section 1.1) et fédération d'identité simplifiée (section 1.2.3) — ont été documentés au moment de leur décision.

1.8.1 Difficultés rencontrées

Chacun des cinq incidents ci-dessous a produit une correction et une leçon transférable.

Indisponibilité liée au démarrage à froid. Le service d'encodage, réduit à zéro instance hors traitement, doit démarrer et charger le modèle lorsqu'une requête arrive après une période d'inactivité. Les mesures rapportées début août ont établi une durée moyenne d'environ vingt-sept secondes, avec un pic à quatre-vingt-quatorze secondes (??) — un tout autre ordre de grandeur que l'estimation initiale, conservée avec la mesure qui la contredit plutôt que corrigée discrètement. Le budget de la sonde de disponibilité a été porté au-dessus du pic observé ; maintenir une instance active en permanence aurait supprimé le problème, pour un coût disproportionné.

Blocage collectif par la limitation de débit. Le mécanisme initial, un seuil unique par adresse IP, bloquait des utilisateurs légitimes : le tableau de bord et l'API partageant une adresse

de sortie commune, l'activité normale de quelques analystes suffisait à franchir le seuil. La correction a introduit la limitation à deux niveaux décrite au chapitre 4, complétée côté applicatif par le compteur indexé sur le défi (section 1.6.3).

État divergent après une application partiellement échouée. Le 08/08/2026, une modification d'infrastructure appliquée sur la recette s'est terminée en échec partiel : une contrainte du fournisseur, non évaluée à la planification, a refusé une partie des changements. L'état local a pourtant enregistré une configuration jamais appliquée, si bien que la planification suivante ne signalait plus aucune différence alors que l'infrastructure réelle en présentait une. L'écart a été détecté par comparaison explicite entre l'état enregistré et les valeurs lues chez le fournisseur, puis corrigé par resynchronisation. Le constat à retenir est factuel : un échec partiel peut laisser l'état local différent de l'état réel, et la planification suivante hérite de cette erreur.

Porte SAST en faux vert. L'action Semgrep utilisée par la chaîne échouait en interne sur une incompatibilité de sévérité, tandis que l'étape apparaissait verte : le 16/08/2026, toutes les tâches de la chaîne étaient en succès alors qu'aucune règle n'avait été évaluée malgré la porte annoncée. Le 19/08/2026, l'action a été remplacée par un appel direct à l'outil en ligne de commande, avec propagation du code de sortie ; la première exécution réelle, numéro 32264602112, a analysé 439 fichiers avec 612 règles, produit 74 constats et terminé avec le code 1. La chaîne en échec prouve ici le rétablissement du contrôle ; elle ne prouve pas la remédiation des 74 constats, dont le triage reste nécessaire.

Capture K23 --- spécification de prise

Vue : historique des exécutions de la chaîne de livraison applicative, filtré sur le flux qui porte la porte d'analyse statique, cadré sur deux lignes voisines et l'en-tête de colonnes.

Dates portées par la capture : 16/08/2026 et 19/08/2026, preuve d'archive.

Contenu probant : la seule chose que ce chapitre ne peut pas écrire — *qu'une porte inerte a l'air de fonctionner*. Ligne du 16/08 : toutes les tâches en succès, alors qu'aucune règle n'a été évaluée. Ligne du 19/08 : la même chaîne en échec après remise en service, exécution 32264602112.

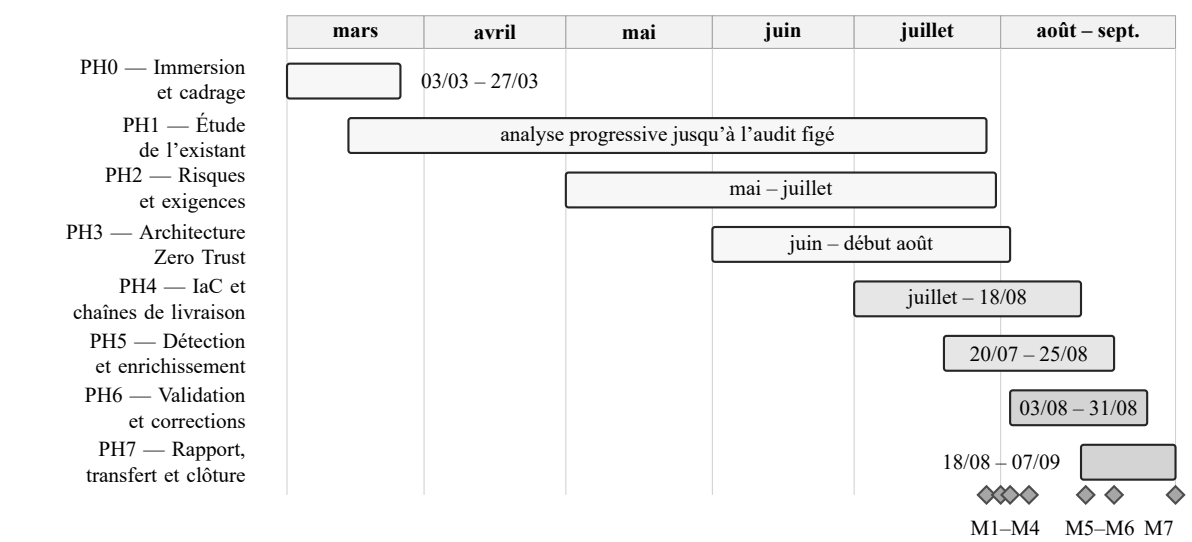
Masquage : nom du dépôt et de l'organisation, courriels des auteurs de *commit*.

FIG. 7. Historique des exécutions de la chaîne de livraison applicative. À y lire, sur deux lignes voisines : le 16/08/2026, toutes les tâches en succès alors que la porte d'analyse statique n'évaluait aucune règle ; le 19/08/2026, la même chaîne en échec après remise en service. Le vert de la première ligne est l'objet de la preuve : c'est lui, et non un journal, qui montre qu'un contrôle inerte est indiscernable d'un contrôle qui passe.

Une règle de sortie qui n'aurait rien gouverné. Le refus par défaut en sortie figurait au plan de correction bien avant d'être posé, et la lecture croisée de la topologie et du paramétrage des charges a montré qu'il aurait été *sans effet*, pour la raison analysée en ?? : le trafic ne passait pas là où la règle se serait appliquée. La correction a donc consisté à déplacer d'abord le trafic, puis à l'observer sept jours, et à n'interdire qu'ensuite (section 1.3.3). La leçon est transférable telle quelle : **un contrôle qui ne voit pas passer le trafic n'est pas un contrôle, c'est une déclaration** — avant de mesurer l'efficacité d'une règle, il faut établir qu'elle est sur le chemin.

1.8.2 Planning de travail

La période de stage s'étend du 3 mars au 7 septembre 2026, et la figure 8 la couvre intégralement. Faute de journal quotidien avant le 29 juillet, ses barres n'affichent pas une fausse précision au jour près ; les losanges M1 à M7 renvoient à des jalons datés par une trace technique.



M1 — 29/07 : audit statique de l'existant figé.
M2 — 01/08 : export int8 rejeté, fp32 conservé.
M3 — 03/08 : restauration chronométrée.
M4 — 07/08 : accueil du locataire ELSON.
M5 — 19/08 : correctifs et scénario de bout en bout.
M6 — 25/08 : date de référence et consolidation des preuves.
M7 — 07/09 : fin du stage et transfert des livrables.

FIG. 8. Planning consolidé du stage, du 03/03/2026 au 07/09/2026. Les trois teintes de barre distinguent le cadrage (PH0–PH3), la construction (PH4–PH5) et la validation et la clôture (PH6–PH7). Les barres sont une reconstruction macroscopique ; les losanges signalent les jalons exactement datés, dont la légende donne la date une à une.

Conclusion du chapitre

L'infrastructure est décrite en code, et la frontière de ce principe est énoncée par six exceptions nommées plutôt que par une revendication d'exhaustivité. La chaîne de livraison compte trois portes bloquantes, cinq familles de contrôles réalisées et trois arbitrages assortis de leur condition de réévaluation ; une seule famille voit son résultat quitter la chaîne, ce qui rend la boucle F6 possible. La chaîne de provisionnement s'arrête volontairement avant l'application, dont l'automatisation exigerait une concentration de privilège que le chapitre 4 refuse, et le tableau de bord ne détient aucune autorisation propre. L'accueil du second locataire isole l'identité, les secrets, les données et, depuis la migration du 11 au 25/08/2026, le réseau ; le stockage reste partagé, faute qu'une seconde migration soit soutenable dans la même quinzaine par un opérateur unique. Deux limites restent assumées : deux charges conservent une sortie managée hors du pare-feu (T14, non conforme), et **l'enrichissement n'alimente pas le score d'incident**.